

Mecanica QA - Gerenciamento Preditivo

Manutenção Preditiva de Trocas de Óleo

Autores:

Denilson Santos

Gabriel Arlisson de Souza Santos Torres

João Arthur Nascimento Mascarenhas

Tony Cleriston Oliveira dos Santos Junior

Vinicius Cerqueira Oliveira

Feira de Santana, 2026

Importação das Bibliotecas

Esse bloco só carrega as ferramentas que serão usadas no resto do notebook. os mais importantes:

- **pandas (pd)** manipula dados em formato de tabela (DataFrame), como um Excel dentro do Python
- **seasonal_decompose** separa uma série temporal em tendência, sazonalidade e ruído
- **Pipeline, SimpleImputer, StandardScaler, GaussianNB** são ferramentas de pré-processamento e um modelo de classificação do scikit-learn

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Carregando o Dataset

Aqui lemos o arquivo CSV com o histórico de trocas de óleo e manutenções.

A coluna Data é convertida para o formato de data e usada como índice da tabela.

Depois o `sort_index()` organiza os registros em ordem cronológica, garantindo que as informações fiquem na sequência correta.

E o `head()` exibe as cinco primeiras linhas para verificar se os dados foram carregados e organizados corretamente.

```
df = pd.read_csv('dataset.csv', parse_dates=['Data'], index_col='Data')
df = df.sort_index()
df.head()
```

	Trocas_Oleo	Manutencao_Motor
Data		
2024-01-01	11.0	2.0
2024-01-02	9.0	4.0
2024-01-03	12.0	1.0
2024-01-04	15.0	8.0
2024-01-05	25.0	10.0

Preenchendo Dias e Valores Faltantes

- A começar pela organização temos o `pd.date_range(...)` que cria todos os dias entre a primeira e a última data do dataset. Depois, `df.reindex(dias)` reorganiza a tabela para garantir que exista uma linha para cada dia, preenchendo com NaN os dias que não estavam no CSV.
- Conversão: `pd.to_numeric()` transforma os dados em números e valores inválidos em vazios.
- Preenchimento: `.interpolate()` estima valores faltantes, enquanto `.ffill()` e `.bfill()` completam os vazios restantes.

```

dias = pd.date_range(df.index.min(), df.index.max(), freq='D')
df = df.reindex(dias)
df.index.name = 'Data'

df['Trocas_Oleo'] = pd.to_numeric(df['Trocas_Oleo'], errors='coerce')
df['Trocas_Oleo'] = df['Trocas_Oleo'].interpolate().ffill().bfill().round()

df['Manutencao_Motor'] = pd.to_numeric(df['Manutencao_Motor'], errors='coerce')
df['Manutencao_Motor'] = df['Manutencao_Motor'].interpolate().ffill().bfill()

df.isna().sum()

      0
Trocas_Oleo  0
Manutencao_Motor  0

dtype: int64

```

Tratando Outliers

Corrige valores absurdos digitados errado (ex: 500 trocas em um dia), limitando o valor máximo aceito.

- **.mean() e .std():** calculam a média e o desvio padrão da coluna
- **média + 3 × desvio:** regra estatística comum (3-sigma) para definir o limite do que é considerado 'normal'
- **.clip(upper=limite):** qualquer valor acima do limite é reduzido até esse limite; a linha continua existindo, só o valor é ajustado

E no print só exibe o limite calculado, para auditoria do usuário

```
media = df['Trocas_Oleo'].mean()
desvio = df['Trocas_Oleo'].std()
limite = round(media + 3 * desvio)
print('limite considerado normal:', limite)

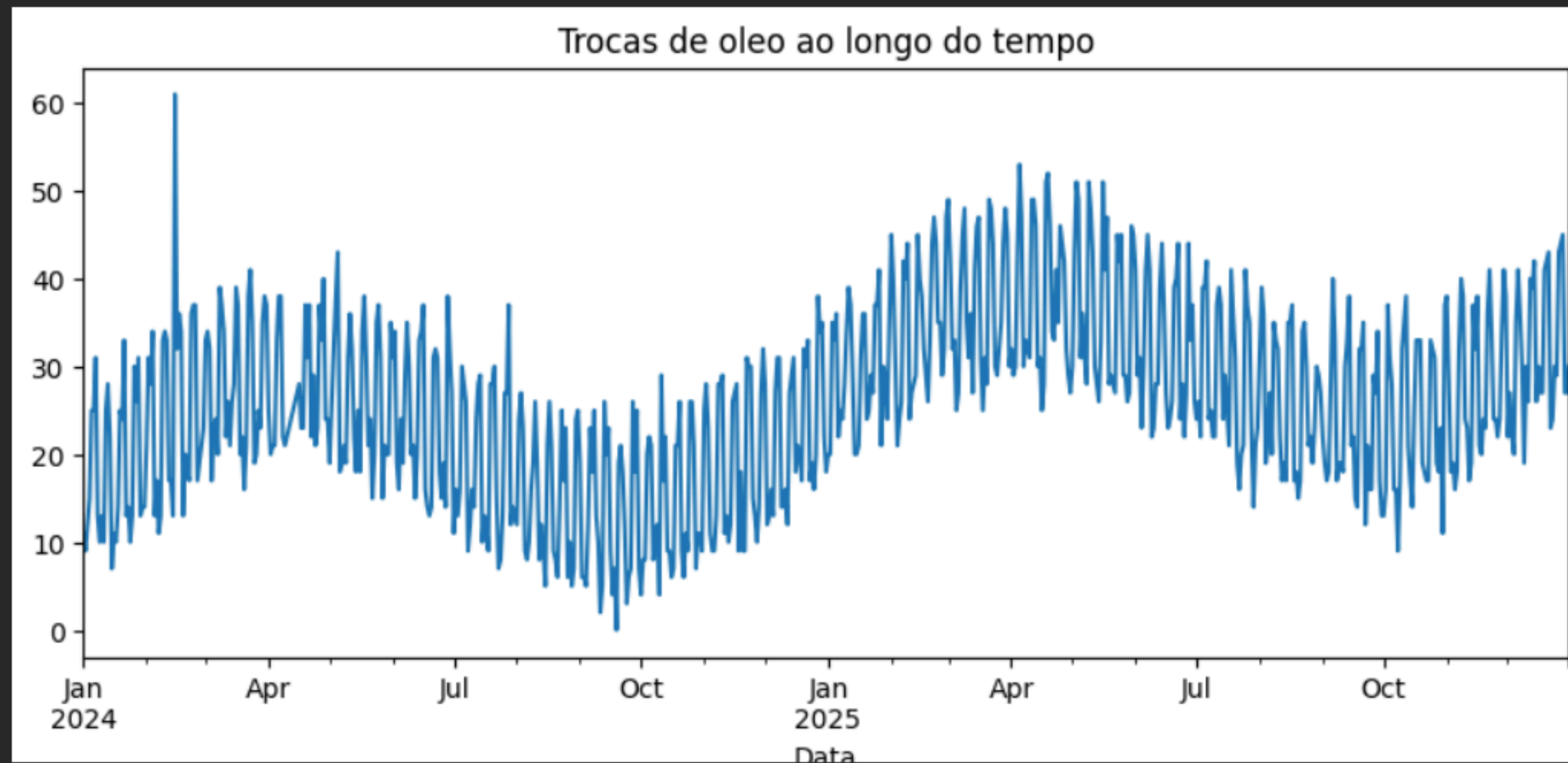
df['Trocas_Oleo'] = df['Trocas_Oleo'].clip(upper=limite)

limite considerado normal: 61
```


Visualizando a Série Temporal

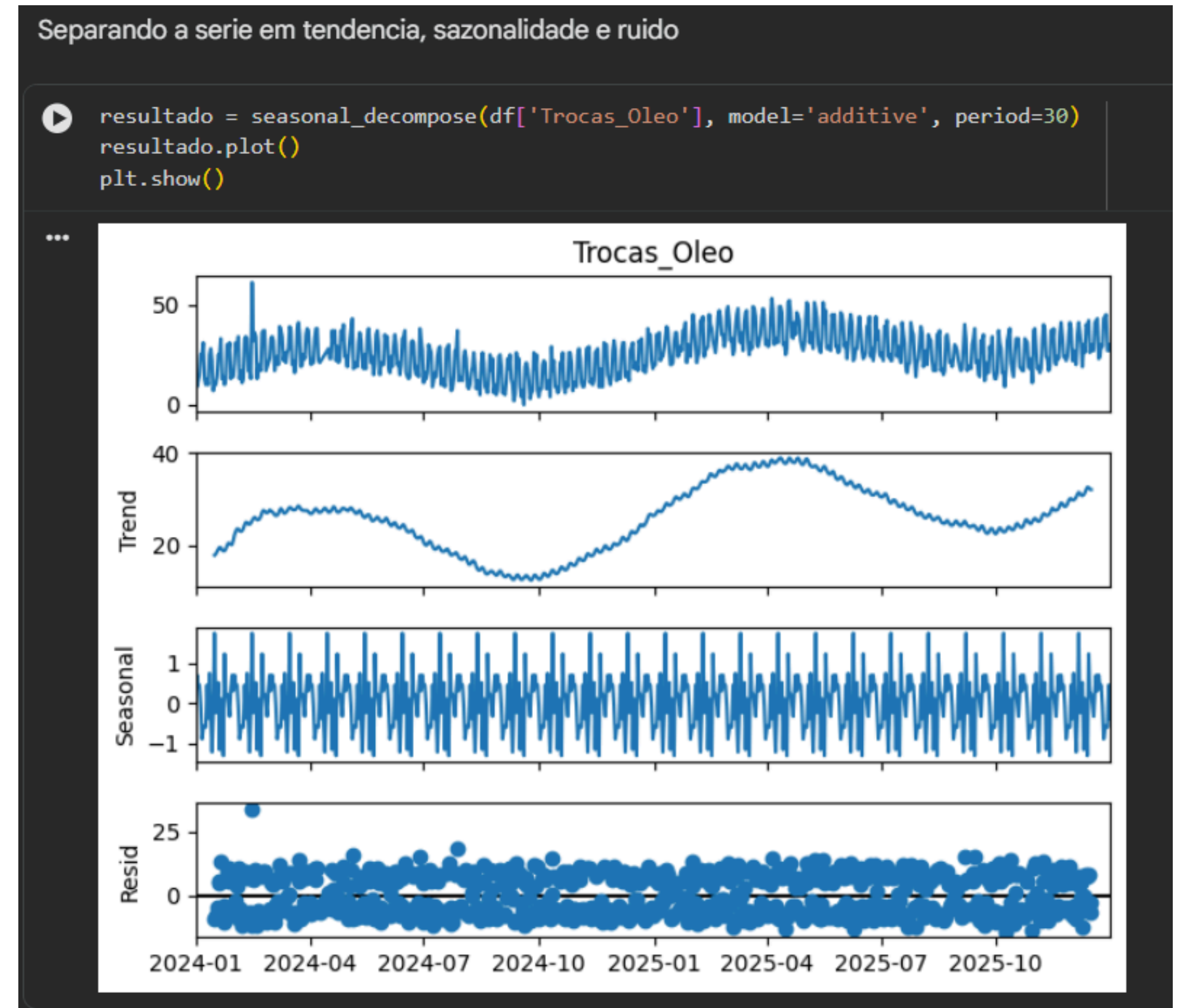
Grafico da serie completa

```
df['Trocas_Oleo'].plot(figsize=(10, 4), title='Trocas de oleo ao longo do tempo')  
plt.show()
```



Decompondo a Série Temporal

- Separa a série em três partes para entender melhor o comportamento dos dados: tendência, sazonalidade e ruído.



Modelo Baseline: Naive

Cria a previsão mais simples possível, só para servir de comparação: a previsão de amanhã é igual ao valor de hoje.

O `.shift(1)` desloca os valores da coluna uma posição para frente, na linha de amanhã ele coloca o valor de hoje

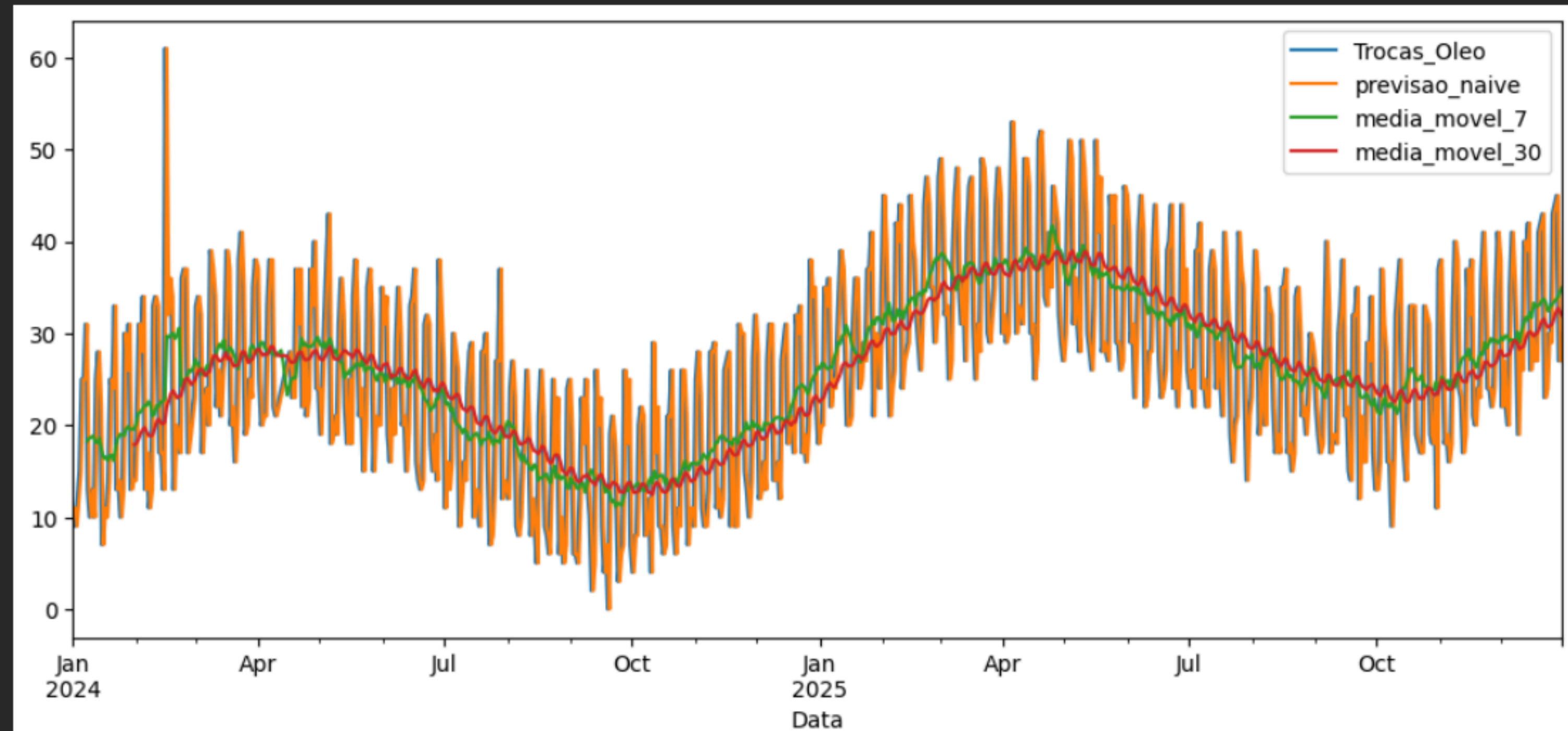
```
df['previsao_naive'] = df['Trocas_Oleo'].shift(1)
```

Segundo modelo simples de comparação: prevê o próximo valor como a média dos últimos 7 ou 30 dias.

```
df['media_movel_7'] = df['Trocas_Oleo'].shift(1).rolling(7).mean()  
df['media_movel_30'] = df['Trocas_Oleo'].shift(1).rolling(30).mean()
```


Comparando Real vs. Previsões

```
df[['Trocas_Oleo', 'previsao_naive', 'media_movel_7', 'media_movel_30']].plot(figsize=(12, 5))  
plt.show()
```



Avaliando o Modelo Naive

Mede o quão boa é a previsão naive comparando com o valor real, usando os últimos 20% dos dados como teste.

- **.iloc[-n_teste:]** — pega as últimas n_teste linhas; como são dados no tempo, o teste precisa ser o período mais recente (não pode ser aleatório)
- **mean_absolute_error** — erro médio absoluto: em média, quanto o modelo erra para mais ou para menos
- **mean_squared_error** — erro quadrático médio: penaliza mais os erros grandes, porque eleva a diferença ao quadrado

```
df_avaliacao = df.dropna(subset=['previsao_naive'])

# Separando 80% treino e 20% teste
n_teste = int(len(df_avaliacao) * 0.2)
df_teste_base = df_avaliacao.iloc[-n_teste:]

y_real_teste = df_teste_base['Trocas_Oleo']
y_previsto_naive = df_teste_base['previsao_naive']

mae_naive = mean_absolute_error(y_real_teste, y_previsto_naive)
rmse_naive = mean_squared_error(y_real_teste, y_previsto_naive)

print(f"Métricas do Modelo Naive no Teste")
print(f"MAE: {mae_naive:.2f} trocas")
print(f"RMSE: {rmse_naive:.2f} trocas")
```

Métricas do Modelo Naive no Teste
MAE: 6.48 trocas
RMSE: 79.36 trocas